

LECTURE 5

The Relational Model

CSC270 · Database Systems

COMSATS University Islamabad, Lahore Campus

Agenda

- Overview of the relational model — what Codd proposed, and why it won
- The notion of a relation: attributes, tuples, degree, cardinality
- Mathematical relations vs. database relations
- The four properties every valid relation must satisfy
- Attributes and domains
- The key hierarchy: superkey, candidate key, primary key, alternate key, foreign key
- Relational database schema vs. database instance
- Standard notation for writing a relation schema

Why the Relational Model Won

- Before 1970, finding “every property managed by branch B003” required already knowing the physical storage path to the data — hard-coded into the application.
- Codd’s insight: a table plus a shared key value (branchNo) answers that question declaratively — you state what you want, and the query language figures out how to get it.
- That separation of “what” from “how” is the foundation Lecture 7’s relational algebra — and later, SQL — are built on.
- Proposed by E. F. Codd in 1970, “A Relational Model of Data for Large Shared Data Banks” — replacing rigid hierarchical and network models that forced hand-coded navigation paths.

The Relational Model, in One Idea

Store data as **tables**. Let mathematical rules — not insertion order, not physical storage — define what a valid table looks like and how tables relate.

Proposed by E. F. Codd, 1970

“A Relational Model of Data for Large Shared Data Banks” replaced hierarchical/network models that forced programmers to navigate hard-coded physical paths.

Branch (branchNo, street, city, postcode)

branchNo	street	city	postcode
B003	163 Main St	Lahore	54000
B005	22 Deer Rd	Karachi	74200
B007	16 Argyll St	Islamabad	44000

Staff (staffNo, name, position, salary, branchNo)

staffNo	name	position	salary	branchNo
SL21	John White	Manager	30000	B005
SG37	Ann Beech	Assistant	12000	B003
SG14	David Ford	Supervisor	18000	B003
SA9	Mary Howe	Assistant	9000	B007
SL41	Julie Lee	Manager	22000	B005

Running example: a property rental agency — Branch, Staff, and the properties each branch manages — carried across Lectures 5–8.

Worked Example: Reading Degree and Cardinality

Branch (branchNo, street, city, postcode)

branchNo	street	city	postcode
B003	163 Main St	Lahore	54000
B005	22 Deer Rd	Karachi	74200
B007	16 Argyll St	Islamabad	44000

- Step 1 — count the columns: branchNo, street, city, postcode → 4 attributes → degree = 4.
- Step 2 — count the rows: B003, B005, B007 → 3 tuples right now → cardinality = 3.
- Compare to Staff: 5 attributes (degree 5), 5 tuples today (cardinality 5) — different relations can have completely different degree and cardinality.
- Cardinality changes the moment a new branch opens; degree only changes if the table's structure itself is altered (ALTER TABLE).

The Notion of a Relation

A relation is a table of values, organized into rows and named columns.

Name — unique within the schema (Staff, Branch)

Attributes — named columns, each with a domain of legal values

Tuples — rows, each holding one value per attribute

Table, relation, or relvar?

Precisely, a relation is a value — one snapshot of tuples. What CREATE TABLE actually makes is a relation variable (relvar), whose value changes as rows are added or removed.

Staff — degree 5, cardinality 5

staffNo	name	position	salary	branchNo
SL21	John White	Manager	30000	B005
SG37	Ann Beech	Assistant	12000	B003
SG14	David Ford	Supervisor	18000	B003
SA9	Mary Howe	Assistant	9000	B007
SL41	Julie Lee	Manager	22000	B005

Degree = number of attributes = 5

Cardinality = number of tuples = 5 (changes on every insert/delete)

Worked Example: A Relvar Changes Over Time

Staff — tomorrow, after HR hires Zara Malik (SG50)

staffNo	name	position	salary	branchNo
SL21	John White	Manager	30000	B005
SG37	Ann Beech	Assistant	12000	B003
SG14	David Ford	Supervisor	18000	B003
SA9	Mary Howe	Assistant	9000	B007
SL41	Julie Lee	Manager	22000	B005
SG50	Zara Malik	Assistant	11000	B005

- The Staff relvar is the same CREATE TABLE structure over time; a Staff relation is just one instant's snapshot of values.
- Today's snapshot: cardinality 5. Tomorrow's snapshot, after the new hire: cardinality 6 — a new relation value, same relvar.
- Degree never changed: still 5 attributes (staffNo, name, position, salary, branchNo) before and after.

Mathematical Relations vs. Database Relations

“Relation” comes from set theory — a subset of a Cartesian product $D_1 \times D_2 \times \dots \times D_n$. A database relation tightens several rules that plain math relations don’t enforce.

	Mathematical relation	Database relation
Tuple order	Significant — $(1,2) \neq (2,1)$	Irrelevant — a relation is a set
Column identity	By position only	By name (branchNo, salary)
Duplicate tuples	Allowed (bag-like in some treatments)	Never allowed — a true set
Component values	Any value, including nested tuples	Must be atomic (single, indivisible)

Atomicity is stricter than it sounds telNo: “042-121-1121, 042-121-1122” looks harmless as a string, but violates atomicity in spirit — the DBMS can’t search or constrain “one of this client’s phone numbers” without parsing it. Fix: a separate ClientPhone(clientNo, telNo) relation, one tuple per number.

Worked Example: Fixing an Atomicity Violation

Before — violates atomicity

ownerNo	name	telNo
CO46	Joe Keogh	042-121-1121, 042-121-1122

- Packing “042-121-1121, 042-121-1122” into one telNo cell looks harmless, but it violates atomicity: the DBMS can’t search, index, or constrain “one phone number” without parsing the string yourself.
- Fix: pull phone numbers into their own relation — ClientPhone(clientNo, telNo) — one tuple per number, each one a first-class, queryable value.
- Trade-off: one extra table in exchange for real atomicity. This exact trade-off is what normalization (Unit 5) teaches you to reason about formally.

Relation Properties — the Four Rules

- 1 Unique name**
Distinct from every other relation in the schema.
- 2 No duplicate tuples**
A relation is a mathematical set — sets cannot repeat an element.
- 3 Tuple order irrelevant**
Rows in any order are the same relation — ORDER BY imposes display order only.
- 4 Attribute order irrelevant**
A value is identified by attribute name, not column position.
- 5 Every value is atomic**
One indivisible value per cell — never a list or nested table.

Recap: Spot the Violation

- Two identical Staff rows get inserted by a buggy script → breaks Rule 2 (no duplicate tuples) — a relation is a true set.
- A clerk records position as “Manager, Assistant” in one cell for a staff member with two roles → breaks Rule 5 (every value atomic).
- A report sorts Branch by city and claims “that’s the real order of the relation” → breaks Rule 3 (tuple order is irrelevant) — only ORDER BY imposes order.
- A spreadsheet import creates two different relations both named “Staff” → breaks Rule 1 (every relation needs a name distinct from every other relation in the schema).

Attributes and Domains

Attribute

A named column representing one property of the thing the relation describes.

Domain

The set of all legal values for an attribute — a reusable data type with business meaning attached.

Shared domains make joins meaningful

Staff.branchNo and Branch.branchNo share the BranchNumbers domain despite matching names — that shared domain is exactly what makes comparing (and later joining, Lecture 8) them meaningful.

Domain definitions used by Staff and Branch

Domain	Meaning	Example values
StaffNumbers	valid staff IDs	SL21, SG37
StaffNames	person names	John White
StaffPositions	valid job titles	Manager, Assistant
Salaries	PKR 8,500–40,000	9000, 30000
BranchNumbers	valid branch IDs	B003, B005

Worked Example: Why Domains Must Match

- Staff.branchNo and Branch.branchNo both draw from the BranchNumbers domain — the same kind of value, so comparing them (Staff.branchNo = Branch.branchNo) is meaningful. This is exactly the condition Lecture 8's join operators rely on.
- Staff.salary (the Salaries domain) and Branch.postcode (a string domain) do not share a domain — comparing salary = postcode is nonsensical even if both happen to be stored in similar-looking columns.
- Quick test: ask “do these two attributes draw values from the same real-world category of thing?” If no, don't compare or join on them — no matter what the raw data type says.

Relational Keys — the Hierarchy

Superkey

Any attribute set that uniquely identifies a tuple — may carry extra attributes



Candidate key

A minimal superkey — remove one attribute and uniqueness breaks



Primary key

The candidate key the designer chooses to identify tuples

`{staffNo}` is a candidate key of Staff: minimal, and unique. `{staffNo, name}` is a superkey but *not* a candidate key — it isn't minimal.

Alternate key

Every candidate key not chosen as primary — still unique, just not the schema's lead identifier.

Foreign key

An attribute set in one relation that reproduces another relation's primary key, to represent a relationship. Staff.branchNo references Branch.branchNo.

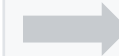
Branch

branchNo

street

city

postcode



Staff

staffNo

name

position

salary

branchNo

Testing for Candidate Keys

A quick test for candidate keys

- 1. Uniqueness** Can two distinct tuples share the same value(s) for K? If yes, K is not even a superkey.
- 2. Minimality** Drop one attribute from K — does uniqueness still hold? If yes, K was not minimal.

Applied to PropertyForRent: `{propertyNo}` is a candidate key; `{propertyNo, street}` is a superkey but not minimal — not a candidate key.

Composite primary keys underline as a group

Not every primary key is one attribute.

Viewing (clientNo, propertyNo, viewDate, comment)

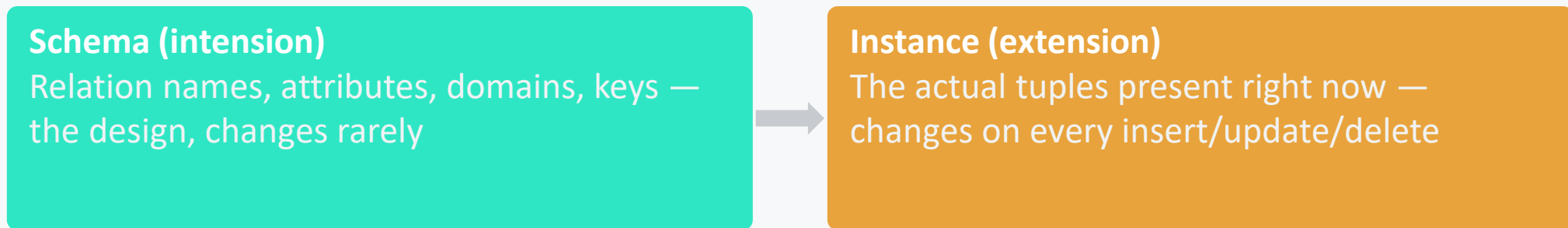
Neither clientNo alone nor propertyNo alone is unique — one client views many properties, one property is viewed by many clients — but the pair is. Convention underlines the whole pair together.

Worked Example: Candidate-Key Test on PropertyForRent

- $K = \{\text{propertyNo}\}$. Uniqueness: can two different properties share a propertyNo? No — it's assigned uniquely. ✓ superkey, and trivially minimal $\rightarrow$ ✓ candidate key.
- $K = \{\text{street}, \text{city}\}$. Uniqueness: could two different properties sit on the same street in the same city (a duplex, say)? Yes, realistically. ✗ not even a superkey.
- $K = \{\text{propertyNo}, \text{ownerNo}\}$. Uniqueness holds (propertyNo alone already guarantees it). Minimality: drop ownerNo — {propertyNo} is still unique. ✗ not minimal — a superkey, but not a candidate key.

Relational Database Schema vs. Instance

A relational database schema is the full structural design: every relation schema (names, attributes, domains, keys) plus every integrity constraint (Lecture 6). The actual rows at any moment are the instance (extension).



By Lecture 8 the rental agency's schema grows to include `PropertyForRent`, `PrivateOwner`, and `Client` — giving the relational-algebra examples something richer to query.

Writing a Relation Schema

```
RelationName (attribute1, attribute2, ..., attributeN)
```

Primary key underlined; foreign keys marked in italics or called out explicitly.

```
Branch (branchNo, street, city,  
        postcode)
```

```
Staff (staffNo, name, position,  
        salary, branchNo)
```

Calling out a foreign key explicitly:

```
Staff (staffNo, name, position, salary, branchNo)  
    Foreign Key branchNo references Branch(branchNo)
```

Memorize this notation cold — every relational-algebra expression from Lecture 7 onward operates on relations named exactly this way.

Worked Example: Writing a Schema with a Foreign Key

- Step 1 — list every attribute: propertyNo, street, city, ownerNo.
- Step 2 — underline the primary key: PropertyForRent(propertyNo, street, city, ownerNo).
- Step 3 — add the foreign-key line naming the attribute and what it references: Foreign Key ownerNo references PrivateOwner(ownerNo).
- Full notation — memorize this cold: PropertyForRent (propertyNo, street, city, ownerNo) Foreign Key ownerNo references PrivateOwner(ownerNo).

From Structure to Correctness

- Lecture 5 gave you the shape of a valid relation: name, attributes, tuples, keys, and schema notation.
- None of that, on its own, stops a bad INSERT — a salary of $-5,000$, or a staff member assigned to a branch that doesn't exist.
- Lecture 6 closes that gap: integrity constraints the DBMS enforces on every single write.

Try It Yourself

- Using Branch(branchNo, street, city, postcode), state the degree and cardinality, then add one new branch and give the new cardinality.
- Test whether {name} alone could be a candidate key of Staff. What real-world fact about names makes your answer NO?
- Write the full schema notation for Viewing(clientNo, propertyNo, viewDate, comment), underlining its composite primary key.
- A colleague claims PropertyForRent needs no foreign keys because “the branch name is stored right there.” In one sentence, explain why that is exactly the design mistake relational modeling teaches you to avoid.

KEY TAKEAWAYS

The Relational Model

- The relational model represents data as named, independent tables linked purely through shared attribute values — no pointers, no nesting.
- A relation is a named set of tuples over fixed named attributes; degree = attribute count, cardinality = current tuple count.
- Database relations tighten mathematical relations: no duplicate tuples, no meaningful order, every value atomic.
- The key hierarchy nests: candidate key is a minimal superkey; primary key is the chosen candidate key; unchosen ones are alternate keys; a foreign key reproduces another relation's primary key.
- A schema is the full structural design; the instance is the actual rows present right now. Schemas are written `Relation(attr1, attr2, ...)` with the primary key underlined.